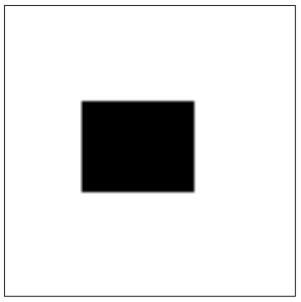
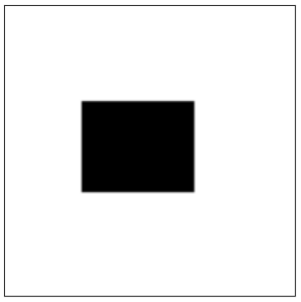
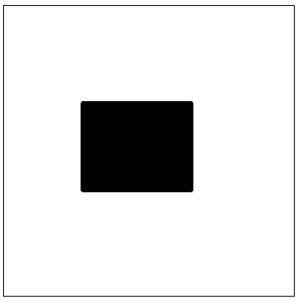
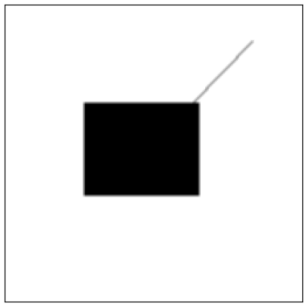
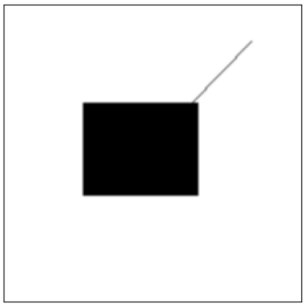
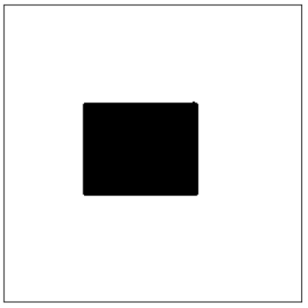
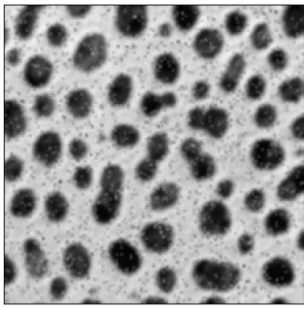
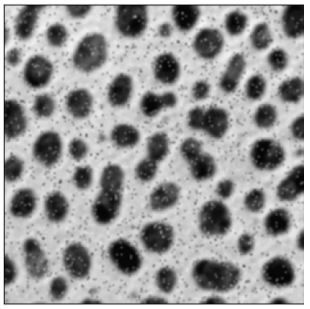
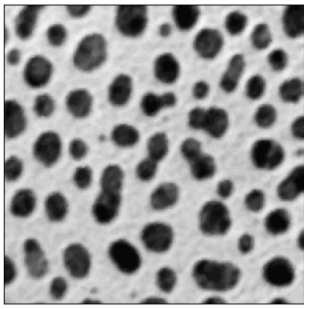
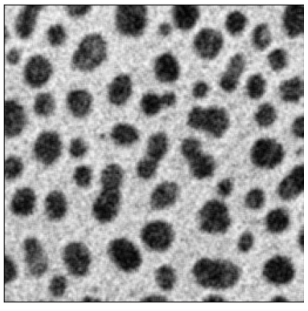
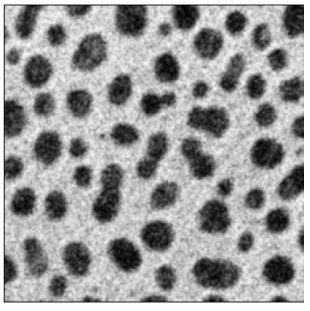
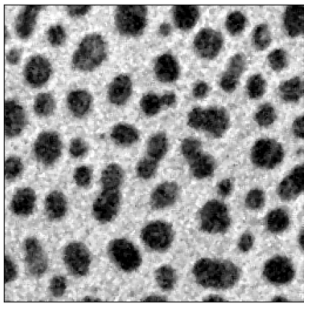
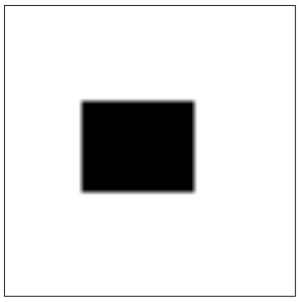
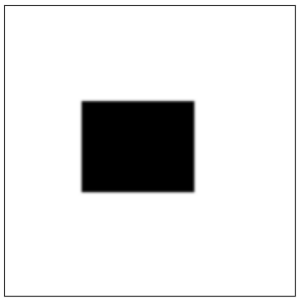
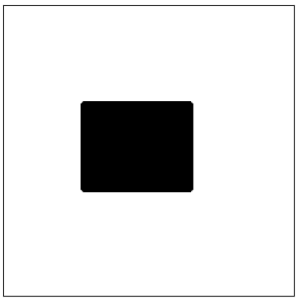
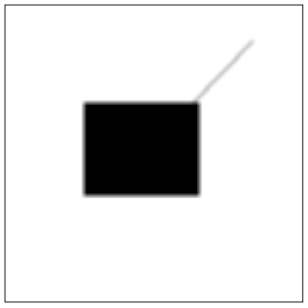
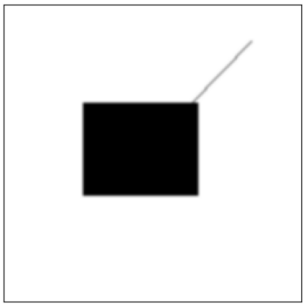
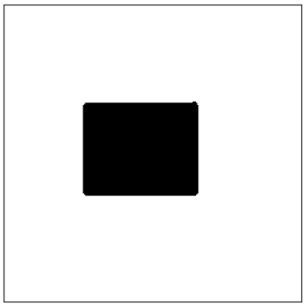
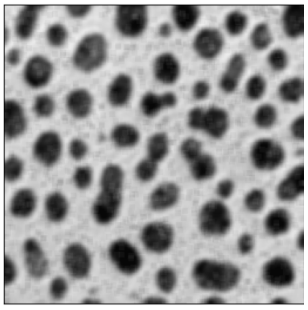
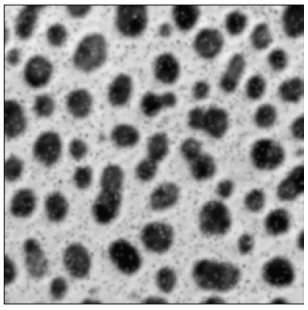
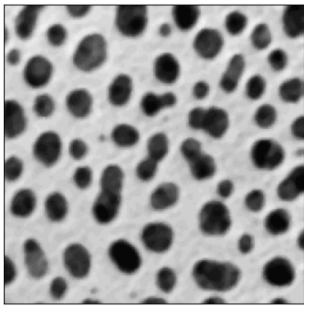
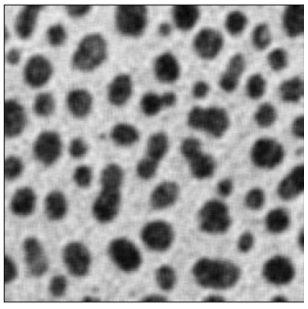
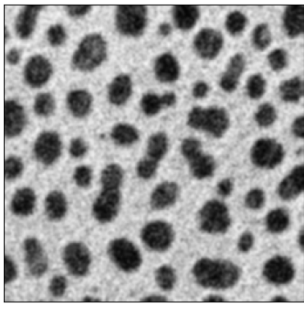
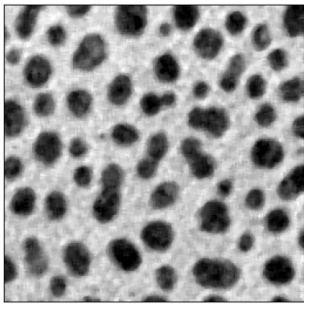
Compte rendu TP note

PARTIE I – EXTRACTION DES COMPOSANTES CONNEXES DANS UNE IMAGE AVEC OPENCV

1. Filtres moyenneur, Gaussien et médian

Avec le code du fichier ***filtrage_Q1_TP.py***, j'ai obtenu les différentes images filtrées suivantes:

Images	Filtre moyenneur 3x3	Filtre Gaussien 3x3	Filtre median 3x3
Carre			
Casserole			
NoisyMultBlobs			
NoisyAddBlobs			

Images	Filtre moyenneur 5x5	Filtre Gaussien 5x5	Filtre median 5x5
Carre			
Casserole			
NoisyMultBlobs			
NoisyAddBlobs			

En analysant les images, on peut remarquer, pour chaque filtre les comportements suivant:

- **Filtre median** : On peut remarquer 2 comportements en fonction de l'image sur laquelle le filtre est appliqué. D'une part, sur les images comme *carre* et *casserole*, le filtre semble arrondir les angles des formes pleines et efface les lignes fines et isolées. D'autre part, sur les images comme *NoisyMultBlobs* et *NoisyAddBlobs* le filtre semble effacer le bruit.
- **Filtre Gaussien** : Ce filtre semble ajouter un léger flou sur la totalité de l'image. En plus de cela, sur les images comme *NoisyMultBlobs* et *NoisyAddBlobs*, le filtre semble réduire la présence de bruit (moins de bruit mais plus épais) mais ne le supprime pas.
- **Filtre moyenneur** : Ce filtre ajoute un flou plus puissant que les deux filtres précédents aux images. ~~En ce qui concerne le bruit,~~ Il semble atténuer l'intensité du bruit mais, comme le filtre Gaussien, il ne semble pas le supprimer.

Les filtres en 5x5 accentuent les effets cités précédemment que l'on peut déjà remarquer en 3x3.

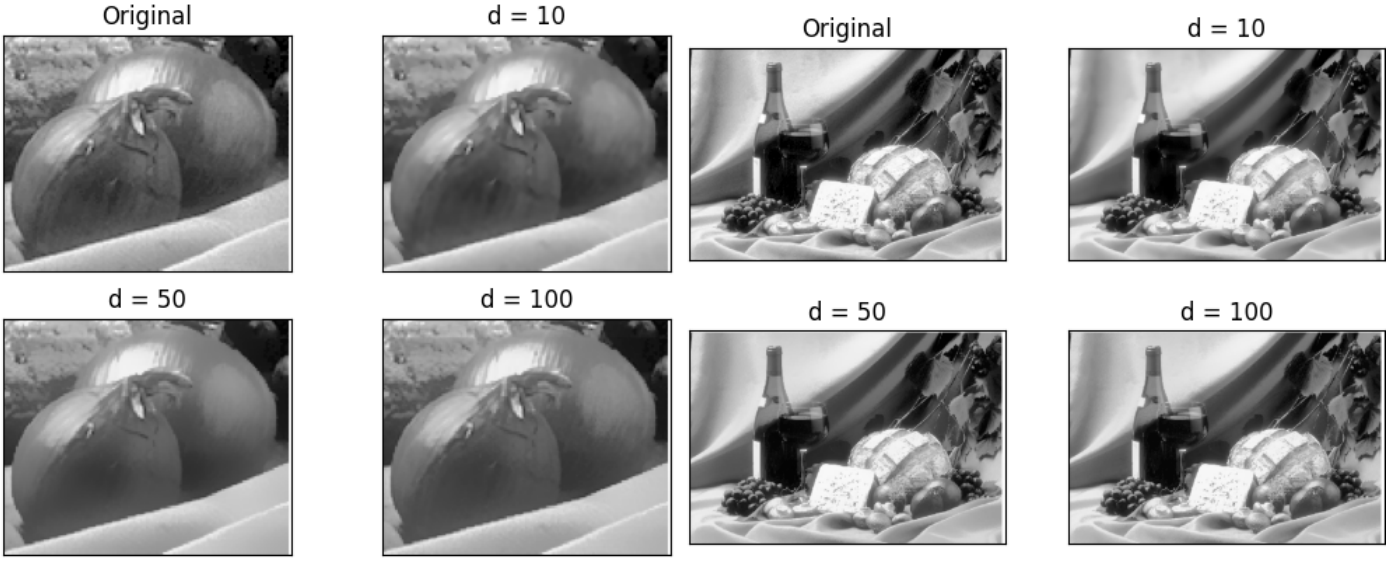
2. Filtre bilatéral

Le filtre bilatéral a pour objectif de "lisser" l'image sur laquelle il est appliqué. C'est-à-dire que le filtre efface les détails présents sur l'image et homogénise les surfaces.

Afin d'observer l'effet des paramètres *d* et *sigma* de la fonction décrivant le filtre bilatéral. J'ai pris plusieurs captures d'écrans en faisant d'abord varier uniquement le paramètre *d* puis uniquement le paramètre *sigma* afin de voir l'impact de chacun sur une image.

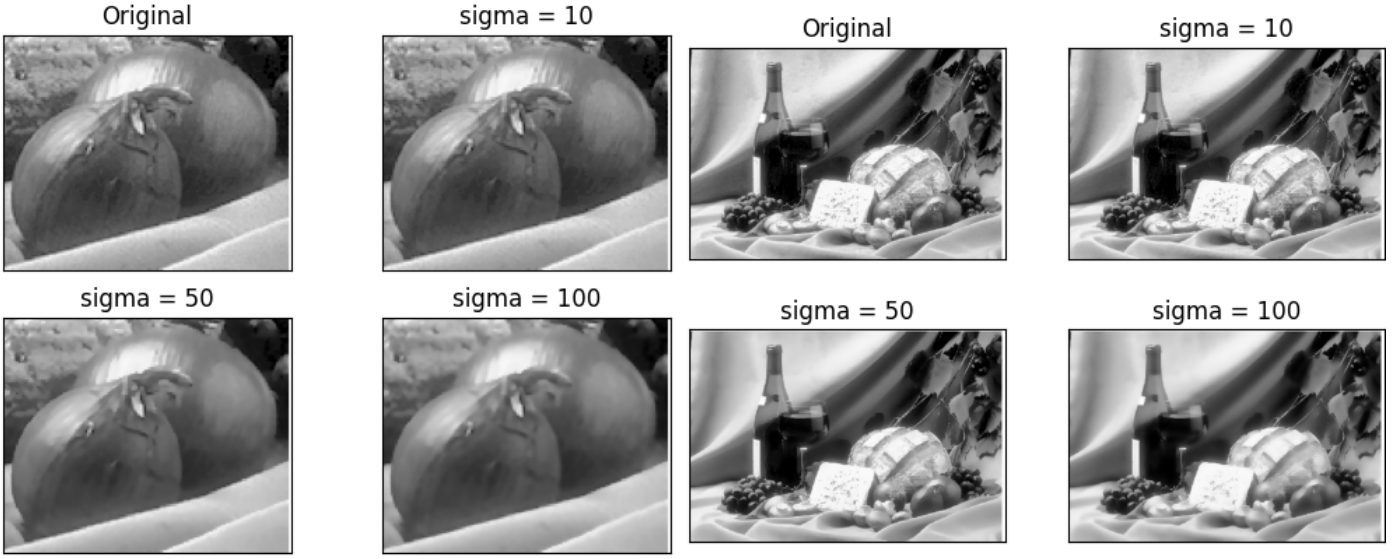
Variation du paramètre *d* :

On peut voir, sur les différentes images ci-dessous, qu'en faisant varier le paramètre *d*, les contours des formes ont l'air d'être préservé alors que les couleurs (ici intensités de gris) et détails à l'interieur des objets (peau de l'oignon, bleu sur le fromage) ont l'air atténués voir totalement dissipés.



Variation du paramètre *sigma*

Sur les images ci-dessous, on peut voir qu'ici les nuances de couleurs ont l'air plus préservé qu'en faisant varier le paramètre *d*. En revanche, les détails ont l'air beaucoup plus attenués notamment en regardant les points sur le fromage ou encore la haie derrière les oignons.



PARTIE II - COMPARAISON DE CONTOURS - DISTANCE D'EDITION - CODE DE FREEMAN

1. L'algorithme de Freeman

Une version de l'algorithme de Freeman que j'ai écrit en python est disponible dans le fichier ***freeman.py***

2. Recherche sur l'algorithme de *Boyer et Moore (77)* et sur l'algorithme de *Wagner et Fischer (74)*

L'algorithme de *Boyer et Moore*

L'objectif de cet algorithme est de trouver dans une chaîne de caractère, appelée "texte" une sous-chaîne de caractère, appelée "clé", ceci en minimisant le nombre de comparaison entre caractères effectués.

Voici ci-dessous une version de l'algorithme en pseudo-code que j'ai construit.

```
entier k = taille de la clé
entier m = taille du texte
entier indice_début_clé = 0
entier indice_comparaison = k

Tant que indice_comparaison != 0 Faire:
    Si clé[indice_comparaison] == texte[indice_comparaison] Alors:
        indice_comparaison = indice_comparaison - 1
    Sinon:
        c = texte[indice_comparaison]
        Si texte[indice_comparaison] est dans clé Alors:
            indice_comparaison = k
            indice_début_clé = indice_début_clé +
                 (indice_comparaison - indice de c dans la clé)
        Sinon:
            indice_comparaison = k
            indice_début_clé = indice_début_clé + k
```

L'algorithme de *Wagner et Fischer*

L'algorithme de Wagner et Fischer permet de calculer la distance d'édition entre deux chaînes de caractères.
Cela signifie que l'algorithme calculé le nombre minimal de modifications nécessaires pour transformer une chaîne de caractères en une autre.

Voici ci-dessous une version de l'algorithme en pseudo-code que j'ai trouvé sur Wikipedia. =

```
entier inf = 1000000000;
entier maxN = 10000;
entier maxD = 100;
tableau cache [maxN][maxD*2+10]; // Initialement initialisé à -1
fonction editDistance(entier i, entier j, chaîne u, chaîne v)
{
    si(i==taille(u))
    {
        renvoyer taille(v)-j;
    }
```

```
    si(j==taille(v))
    {
        renvoyer taille(v)-i;
    }
    si(valeur_absolue(i-j)supmaxD)
    {
        renvoyer inf;
    }
    entier res=cache[i][i-j+maxD];
    si(resSup0)
    {
        renvoyer res;
    }
    res = inf;
    res = min(res, editDistance(i+1, j, u, v)+1);
    res = min(res, editDistance(i, j+1, u, v)+1);
    si(u[i]==v[j])
    {
        res=min(res, editDistance(i+1, j+1, u, v));
    }
    sinon
    {
        res=min(res, editDistance(i+1, j+1, u, v)+1);
    }
    cache[i][i-j+maxD]=res;
    renvoyer res;
}
```

Application au code de Freeman et limites

Le code de Freeman étant la caractérisation du contour d'une forme, on peut les utiliser pour comparer des formes entre elles.
Pour cela, on pourrait faire en sorte de transformer la sortie du programme *freeman.py* en une chaîne de caractère et utiliser les algorithmes décrit ci-dessus pour effectuer certaines comparaisons.

En utilisant l'algorithme de Boyer et Moore, on pourrait essayer des sous-chaînes similaires dans les codes de Freeman. Cela permettrait de savoir si deux formes ont une partie de leur code de freeman en commun et ainsi savoir lsi es deux formes ont des parties identiques.
Malheureusement pour connaître toutes les parties similaires entre 2 formes avec des codes de taille *n* et *m*, il faudrait tester chaque sous-chaîne de taille 1 à *n* sur le code de taille *m*. Pour des formes de grande taille, le temps de calcul deviendrait rapidement trop grand.

Si on utilisait l'algorithme de Wagner et Fischer, on pourrait comparer deux codes de Freeman entre eux. Une valeur faible indiquerait un nombre faible de transformation pour passer d'une forme à l'autre et donc une forte similarité entre les deux formes.
On peut voir certains cas où l'algorithme ne permettrait pas une comparaison correcte. Par exemple, si on essayait de comparer les code de Freeman d'une même forme mais un pris dans le sens horaire et l'autre dans le sens anti-horaire. Dans ce cas, l'algorithme devrait effectuer beaucoup de transformation. Décrivant ainsi une similarité faible entre les formes alors qu'elles sont par définition identiques.

3. Etude de l'algorithme de *Wagner et Fischer (74)*

a.

En utilisant le programme écrit dans la Partie II-1., j'ai obtenu les différents codes de Freeman suivants:

Image	P0	Code de Freeman
xa0_1	(13, 1)	[5, 4, 4, 4, 4, 5, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 3, 4, 4, 4, 4, 3, 4, 4, 4]
xa0_2	(10, 1)	[5, 4, 5, 4, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 3, 4, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
xa0_3	(14, 1)	[5, 4, 4, 4, 4, 4, 5, 6, 0, 0, 0, 0, 0, 7, 0, 0, 0, 1, 0, 0, 0, 0, 1, 3, 4, 4, 3, 4, 4, 4, 4]
xa1_1	(16, 1)	[5, 4, 5, 4, 5, 0, 0, 0, 1, 0, 1, 1, 4, 4]
xa7_1	(6, 1)	[0, 0, 0, 0, 0, 0, 0, 0, 7, 5, 4, 4, 4, 5, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
xa7_2	(4, 1)	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 7, 5, 4, 4, 4, 4, 5, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 2, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]
xt0_1	(14, 1) .	[5, 4, 4, 4, 4, 5, 4, 4, 6, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 3, 4, 3, 4, 4, 4, 4, 4]
xt1_1	(19, 1)	[5, 4, 4, 5, 4, 4, 4, 4, 5, 4, 4, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 4, 4]
xt1_2	(15, 1)	[5, 5, 6, 7, 1, 2, 1, 3]
xt7_1	(3, 1)	[0, 0, 0, 0, 0, 0, 7, 0, 0, 0, 0, 0, 7, 6, 0, 0, 1, 1, 0, 0, 4, 4, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4]

Distance d'édition sans pondération

En suivant la documentation du programme, j'ai obtenu les résultats suivants en mettant une pondération de 1 pour la substitution, insertion et suppression. D'après la documentation, il s'agit donc de la distance de Levenshtein.

	xt0_1	xt7_1	xt1_1	xt1_2
xa7_1	0.5750	0.3250	0.6250	0.9000
xa1_1	0.7059	0.7949	0.4615	0.7143
xa0_1	0.2059	0.5641	0.4839	0.8387
xa0_2	0.3143	0.4359	0.5714	0.8571
xa7_2	0.6170	0.2979	0.6596	0.9149
xa0_3	0.2647	0.5641	0.4194	0.7742

Distance d'édition evec un coût de substitution valant 2

Dans le tableau ci-dessous se trouve les résultats que j'ai obtenu en mettant un krk = 2. D'après la documentation, il s'agit d'une pondération de 2 de la substitution.

	xt0_1	xt7_1	xt1_1	xt1_2
xa7_1	0.7500	0.4250	0.7500	1.0000
xa1_1	0.8235	0.9487	0.4615	1.0000
xa0_1	0.2647	0.6154	0.6129	0.9355
xa0_2	0.3143	0.5128	0.7714	0.9429
xa7_2	0.7447	0.3404	0.7447	1.0000
xa0_3	0.3824	0.7179	0.4839	0.8065

En comparant les deux tableaux, on peut remarquer tout d'abord que les cases qui contiennent les valeurs les plus faibles représentent une forte similarité entre les formes contenues dans les images correspondant à la colonne et à la ligne de cette case. (exemple: xa0_1 et xt0_1 ont une distance de Levenshtein de 0.2647, ces deux formes sont donc assez similaires l'une de l'autre)
Ensuite, on peut aussi remarquer que certaines valeurs ne sont pas modifiées d'un tableau à l'autre. Cela signifie qu'aucune substitution n'est effectuée pour passer d'une image à l'autre.